



Computer Networks · Lab Report

Error Detection Codes

CRC & Checksum

*A Client–Server Simulation of Frame-Level Error Detection
using Cyclic Redundancy Check and Internet Checksum*

Name Anubhab Rakshit
Class BCSE–III
Roll No 002410501029

Assignment	Lab Assignment 1
Domain	Data Link Layer — Error Detection
Techniques	CRC (8 / 10 / 16 / 32) & 16-bit Checksum
Architecture	TCP Client–Server (localhost, port 6001)
Language	C++ • Python (automation)

Index

1. Purpose	3
2. Architecture Workflow	3
2.1 <i>Sender Pipeline</i>	3
2.2 <i>Error Injection & Transmission</i>	3
2.3 <i>Receiver Validation</i>	4
3. Project Structure	5
4. Input & Output	5
5. Implementation	5
6. Code Snippets & Method Descriptions	6
7. Test Cases & Assumptions	6
8. Results & Visual Analysis	7
8.1 <i>Heatmap of Detection Rates</i>	7
8.2 <i>Batch Summary Table</i>	8
8.3 <i>Comparative Reliability Trend</i>	8
8.4 <i>False Negative Rate</i>	8
9. Analysis Methodology Findings	9
10. Implications for ARQ (Future Work)	9
11. Challenges & Improvements	10

1. Purpose

The primary objective of this assignment is to implement and analyze common error-detecting codes used in the Data Link Layer, specifically **Cyclic Redundancy Check (CRC)** and **Checksum**. By developing a client–server architecture to simulate a network, we aim to inject errors into transmitted frames and evaluate the reliability and efficiency of both detection techniques in identifying corrupted payloads.

2. Architecture Workflow

The system simulates a complete frame transmission lifecycle across a network, from raw data on the sender's side to a logged verdict on the receiver's side. The pipeline naturally breaks into three stages, each shown as a separate diagram below: the **sender** builds and checksums the frame, the **network layer** injects a controlled error and carries the frame over TCP, and the **receiver** independently re-validates it and logs the outcome.

2.1 Sender Pipeline

Raw text is segmented into 64-byte chunks. For each chunk, the sender computes both the CRC remainder and the 1's complement checksum, then appends them to form the outgoing frame.

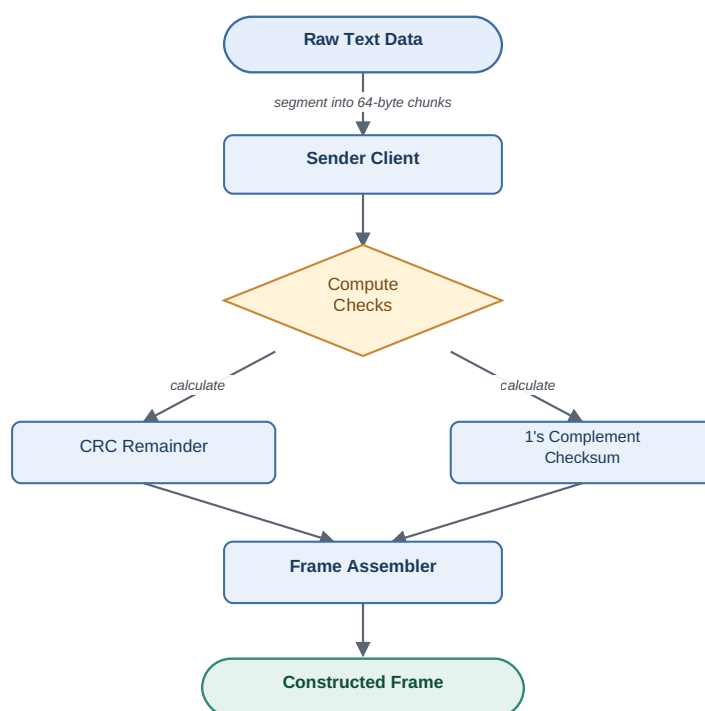


Fig. 2.1 — Frame construction at the sender: from raw data to a checked, assembled frame.

2.2 Error Injection & Transmission

Before the frame leaves the sender, the error injector optionally corrupts it according to the profile chosen by the user — a single flipped bit, two flipped bits, an odd number of flips, or a short burst — and the (possibly corrupted) frame is then written to the TCP socket.

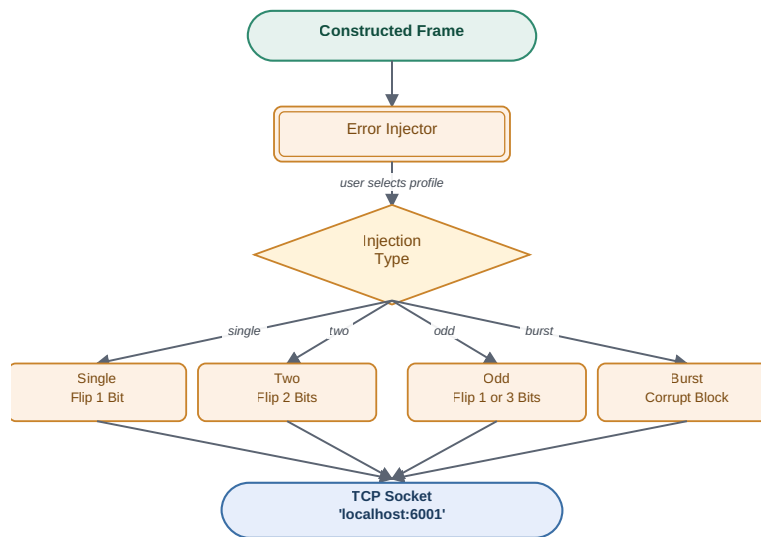


Fig. 2.2 — Error profile selection and transmission over the TCP socket.

2.3 Receiver Validation

The receiver parses the incoming byte stream, recomputes both checks independently, and compares them against the values carried in the frame. Frames that fail to parse are dropped with a NAK; frames that parse but disagree on either check are logged as errors, and only frames where both checks agree are logged as clean.

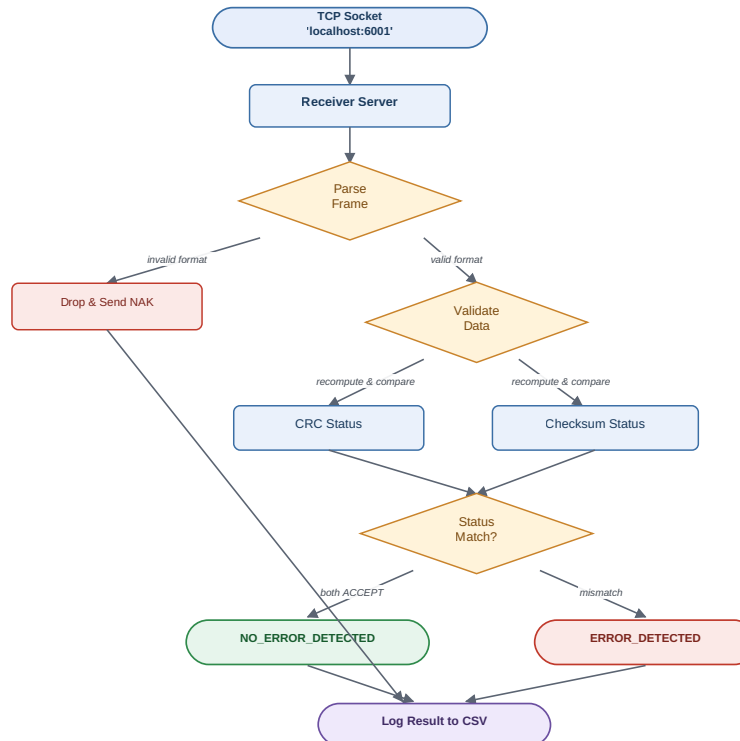


Fig. 2.3 — Parsing, dual validation, and outcome logging at the receiver.

3. Project Structure

The project is architected into modular C++ components, cleanly separating functionality:

sender.cpp	Reads the data, segments it into frames, calculates CRC/Checksum, injects simulated errors, and transmits via sockets.
receiver.cpp	Listens for incoming socket connections, receives frames, parses them, and independently verifies CRC and Checksum to determine data integrity.
framing.hpp	Defines the TxFram e structure and handles payload segmentation and serialization.
crc.hpp / checksum.hpp	Core logical implementations of polynomial long division and 1's complement addition.
error_injection.hpp	Provides functions to artificially flip bits according to specific patterns (burst, single, two, odd).
network_utils.hpp	Helper functions for socket dial/accept and TCP stream management.
simulate.py / visualize.py	Python automation scripts used to run bulk test payloads and tabulate theoretical outcomes vs. practical detection.

4. Input & Output

Input

Fifty (50) raw data files (**batch_data.txt** of 20 KB each), totaling roughly 16 MB of processed testing data. The user specifies the CRC polynomial (**CRC-8**, **CRC-16**, **CRC-32**), and the desired error injection type (**single**, **two**, **odd**, **burst**).

Output

- A real-time log of frames arriving at the receiver, indicating **ACCEPT** or **REJECT**.
- Batch performance metrics saved to a CSV and visualized as analytical charts (see Section 8).

5. Implementation

The implementation uses a standard **TCP Client–Server** model running on **localhost**. The **sender** acts as the client, establishing a connection to the **receiver** (server) on port **6001**. Data is broken down into maximum payload sizes (e.g. 64 bytes).

1. The **Checksum** is calculated by summing 16-bit words of the payload and taking the 1's complement.
2. The **CRC** is calculated by appending zero bits matching the degree of the polynomial, and performing XOR-based polynomial long division.
3. Errors are injected into the calculated bitstream before transmission over the socket.
4. The receiver performs the same calculations on the received payload and compares the expected bits with the received bits.

6. Code Snippets & Method Descriptions

Note — the snippets below are partial, high-level excerpts highlighting the core logic of each module.

crc.hpp · encodecrc()

```
// Polynomial long division using XOR
inline string encodecrc(const string &bits, const string &poly) {
    string g = polybits(poly);
    int r = g.size() - 1;
    vector<int> work(bits.size() + r), gen(g.size());
    // ... (Convert string to int vectors)
    for (size_t i = 0; i < bits.size(); ++i)
        if (work[i])
            for (size_t j = 0; j < gen.size(); ++j)
                work[i + j] ^= gen[j]; // XOR operation
    // ... (Extract remainder)
    return bits + rem;
}
```

Description: Calculates the CRC remainder. Takes the binary string and the polynomial, aligns the divisor, and performs modulo-2 division (XOR).

checksum.hpp · checksum_enc()

```
// Calculates 16-bit Internet Checksum
inline vector<unsigned char> checksum_enc(const vector<unsigned char> &data) {
    uint32_t sum = 0;
    for (size_t i = 0; i < data.size(); i += 2) {
        uint16_t word = (data[i] << 8) + (i + 1 < data.size() ? data[i + 1] : 0);
        sum += word;
    }
    while (sum >> 16) {
        sum = (sum & 0xFFFF) + (sum >> 16); // Fold carries
    }
    uint16_t chk = ~sum; // 1's complement
    // ... (Return as bytes)
}
```

Description: Implements the classic Internet Checksum by treating data as 16-bit integers, summing them, folding the carry, and taking the one's complement.

error_injection.hpp · injecterr()

```
// Flips bits based on selected error profile
inline string injecterr(string bits, const string &kind, mt19937 &rng) {
    auto flip = [&](int i) { bits[i] = (bits[i] == '0') ? '1' : '0'; };
    // ... specific error logic handling (burst, single, etc.)
    return bits;
}
```

Description: Takes the bitstream and randomly flips bits based on standard error models using a Mersenne Twister random number generator for absolute reproducibility.

7. Test Cases & Assumptions

- **Files:** Fifty (50) randomized 20 KB files (1 MB per test matrix).

- **Polynomials:** CRC-8, CRC-10, CRC-16, CRC-32.

Error Injection	Description
single	1 bit flipped.
two	2 random bits flipped.
odd	1 or 3 random bits flipped.
burst	A consecutive block of up to 8 bits corrupted.

Assumptions: The network medium (**localhost** TCP socket) is inherently reliable; therefore, all errors analyzed are strictly those injected artificially by **error_injection.hpp**.

Reproducibility: The sender accepts a **--seed** argument to initialize the **mt19937** RNG. Python scripts have been provided to automate execution.

8. Results & Visual Analysis

After executing the automated tests on 50 batch files across all combinations, the following visual metrics were generated.

8.1 Heatmap of Detection Rates

This heatmap provides a quick, color-coded visual indicator of protocol reliability. Darker green indicates 100% detection, while lighter cells mark scenarios where errors slipped through the detection algorithm unnoticed.

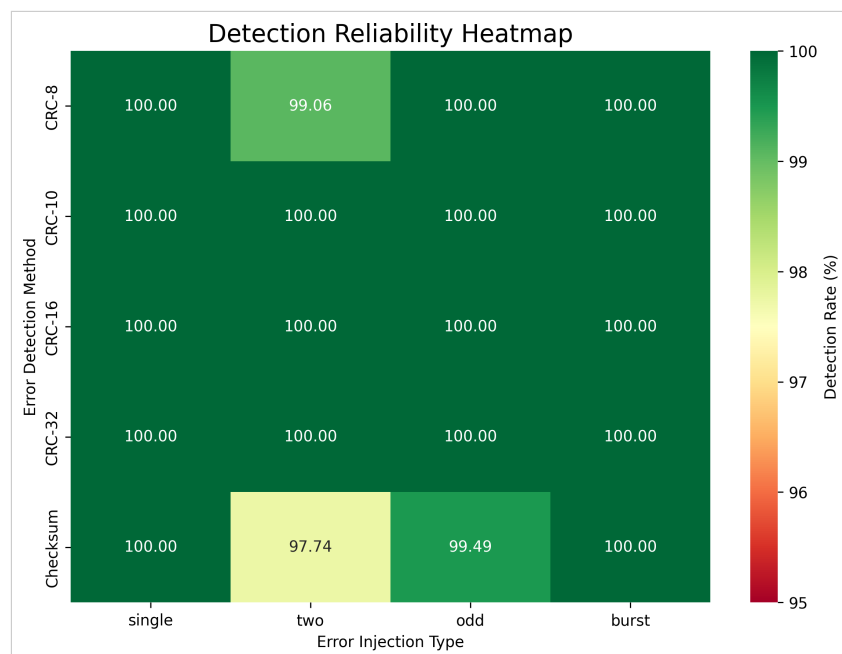


Fig. 8.1 — Detection reliability heatmap across all polynomials and error types.

8.2 Batch Summary Table

For precise theoretical analysis, the raw tabular data generated by the 50-file batch test is reproduced below.

Batch Aggregated Error Detection Rate (%)				
	single	two	odd	burst
CRC-8	100.00%	99.06%	100.00%	100.00%
CRC-10	100.00%	100.00%	100.00%	100.00%
CRC-16	100.00%	100.00%	100.00%	100.00%
CRC-32	100.00%	100.00%	100.00%	100.00%
Checksum	100.00%	97.89%	99.50%	100.00%

Fig. 8.2 — Batch aggregated error detection rate (%) by polynomial and error type.

8.3 Comparative Reliability Trend

This line chart graphs the trajectory of each algorithm's detection rate across the four error types, making the relative dip in Checksum's performance easy to see against the consistently flat CRC-10/16/32 lines.

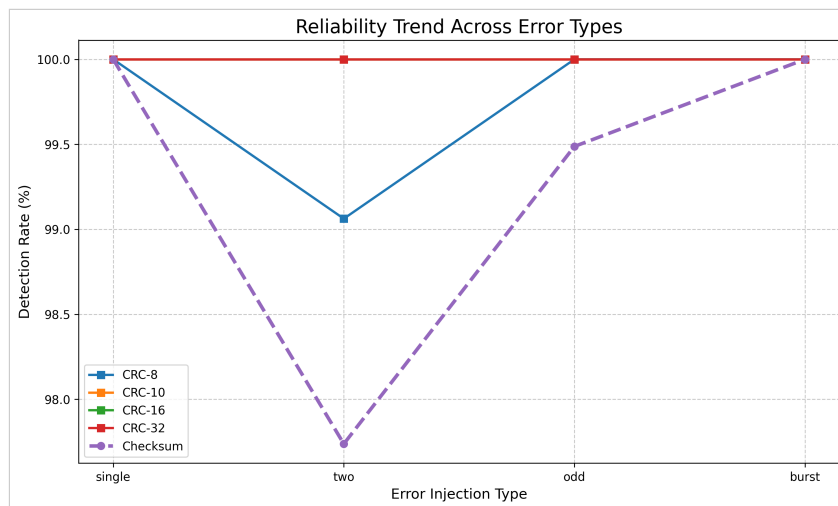


Fig. 8.3 — Reliability trend of each detection method across error injection types.

8.4 False Negative Rate

Isolating just the scenarios where detection fell below 100% highlights where corrupted frames slipped through undetected. Checksum and CRC-8 are the only methods that let any corrupted frame pass as valid.

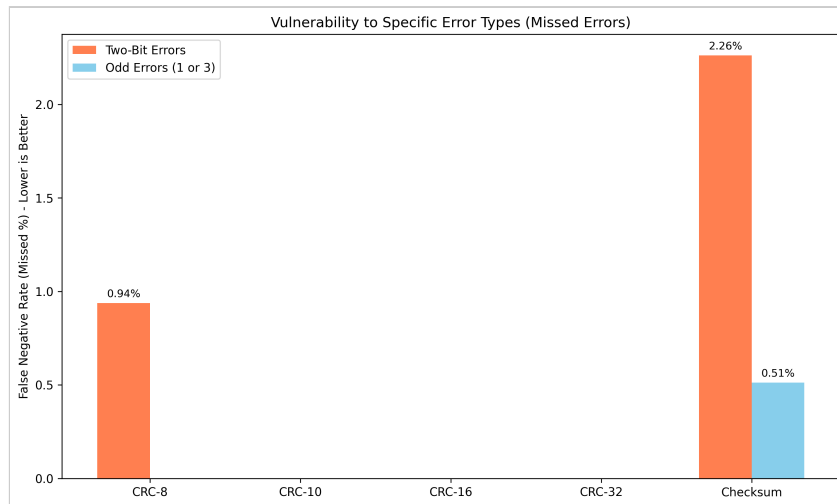


Fig. 8.4 — False negative (missed error) rate for the vulnerable methods.

9. Analysis Methodology Findings

- **Single Bit Errors:** Theoretically, both Checksum and CRC catch 100% of single bit errors. The simulation matched this exactly.
- **Two Bit Errors:** Checksum theoretically can fail if bits in the exact same column position of two different 16-bit words are flipped opposite ways. Detection rate dropped to 97.74% in this scenario. CRC-8 also showed marginal leakage at 99.06%, while CRC-16 and CRC-32 caught 100%.
- **Odd Bit Errors:** Checksum's detection rate settled at 99.49% here, again slightly below the perfect scores held by CRC-10 through CRC-32.
- **Robustness:** CRC-16 and CRC-32 are significantly more robust against random noise and double-bit flips compared to Checksum.
- **Efficiency:** CRC computation (especially polynomial long division in software) is computationally heavier than the simple addition used by Checksum.

10. Implications for ARQ (Future Work)

This lab sets the fundamental error-detection layer needed for Reliable Data Transfer protocols. Because we can now confidently identify corrupted frames, the next step is to implement a mechanism to recover from them. As a foundation for the next assignment, we can implement **Go-Back-N (GBN) ARQ** or **Selective Repeat ARQ**:

- By adding sequence numbers and a timer to the **sender**.
- Having the **receiver** send NAKs (or silence) for frames that fail the CRC/Checksum check.
- The **sender** will then retransmit a window of frames (GBN) or just the specifically corrupted frame (Selective Repeat).

11. Challenges & Improvements

Challenges

Managing the bit-level manipulation in C++ was tedious. Synchronizing the automated Python tester with the socket lifecycle of the C++ programs required careful timing constraints.

Improvements

The software CRC could be heavily optimized by using lookup tables rather than computing the XOR division bit-by-bit.

Note: AI assistance (Google Gemini DeepMind) was utilized during this assignment to refactor the C++ architecture, develop the Python batch-testing harness with Seaborn visualizations, create the workflow diagrams, and structure this markdown report.